

Programming and Data Analysis with Python in the Age of AI

A Systematic Introduction for Absolute Beginners

Summer Semester — Undergraduate Course

2026-06-27

How to use this document

This is a course reader. It assumes **you have never written a line of code before**. We use Python as the vehicle for learning programming itself, so most ideas here (**variables**, loops, functions) carry over to any other language.

A few conventions:

- **Grey boxes are code.** You can type them yourself and run them.
- Lines that begin with `#` are *comments* — notes for humans, ignored by the computer.
- A comment like `# => 5` shows the **expected result** of the line above it.
- Boxes labelled “**Python vs JavaScript vs C#**” compare Python to two other popular languages. You can safely skip these on a first read; they are there for context and for students who have seen other languages.

Running the code

The code in this reader is real Python. To run it you need Python installed (see Section 2). The data-analysis and plotting sections (Section 17 onward) also need three extra packages:

```
pip install numpy pandas matplotlib
```

If you are reading the rendered HTML/PDF, the outputs you see were produced by actually running the code, so what you read is what you would get.

1 What is programming?

A **computer** is a machine that does exactly what it is told, extremely fast, and never gets bored. The catch is the phrase *exactly what it is told*. A computer has no common sense. If you tell it to do something slightly wrong, it will do that wrong thing perfectly, a billion times.

Programming is the craft of writing down instructions precisely enough that a computer can follow them. A **program** is just a **list** of such instructions, saved in a file. A **programming language** (like Python) is a set of words and grammar rules that sit between human ideas and the computer's native language of 1s and 0s.

Think of a recipe:

1. Take two eggs.
2. If you have milk, add a splash.
3. Beat for 30 seconds.
4. Repeat step 3 until the mixture is smooth.

A recipe already contains every idea you will learn in this course:

Recipe idea	Programming idea	Where we cover it
"two eggs", "a splash of milk"	Values and variables	Section 4
"If you have milk..."	Conditionals	Section 9
"Repeat until smooth"	Loops	Section 11
"Beat for 30 seconds" (a named step)	Functions	Section 12
"Take two eggs" (ingredients list)	Data structures	Section 10

The rest of this document teaches each of these, one at a time, in Python.

1.1 Why Python?

Python was designed to be **readable**. Its code looks almost like English, it hides a lot of fussy machinery, and it has an enormous collection of free libraries for science, data, and **AI**. That is why it is the most common first language taught at universities and the default language of data analysis and machine learning.

The trade-off: Python is slower than languages like C# or C++ for raw number crunching. In practice this rarely matters, because the heavy libraries we use (**NumPy**, **Pandas**) are themselves written in fast C under the hood.

2 Setting up Python

You need two things: the Python **interpreter** (the program that runs your programs) and a place to **write** code.

1. **Install Python.** Download it from python.org or, for data science, install the Anaconda distribution, which bundles Python plus the data libraries.
2. **Choose an editor.**
 - *VS Code* — a general-purpose code editor (what this course uses).

- *Jupyter Notebook* — runs code in cells and shows results inline; great for data analysis and for documents exactly like this one.

2.1 Three ways to run Python

(a) **The REPL (interactive prompt).** Type `python` in a terminal and you get a `>>>` prompt where each line runs immediately. Great for experiments.

```
>>> 2 + 2
4
>>> print("hello")
hello
```

(b) **A script file.** Put your code in a file ending in `.py`, e.g. `hello.py`, then run it:

```
python hello.py
```

(c) **A notebook / Quarto cell.** Code lives in “cells” and the output appears right below. This reader is itself such a document.

i Python vs JavaScript vs C#: how programs run

- **Python** is *interpreted*: you run the source file directly.
- **JavaScript** is also interpreted, usually inside a web browser or Node.js.
- **C#** is *compiled* first into an intermediate form, then run. You typically build the project before running it.

For a beginner, “interpreted” simply means: fewer steps between writing and seeing a result — one reason Python is friendly to learn.

3 Your first program

The tradition is to make the computer say hello.

```
print("Hello, world!")
```

```
Hello, world!
```

`print` is a **function** — a built-in command that displays whatever you put in its parentheses. The text in quotes is a **string** (a piece of text). That one line is a complete, working program.

i Python vs JavaScript vs C#: hello world

```
# Python
print("Hello, world!")
```

```
// JavaScript
console.log("Hello, world!");
```

```
// C#
using System;
class Program {
    static void Main() {
        Console.WriteLine("Hello, world!");
    }
}
```

Notice how much *ceremony* C# requires before the interesting line. Python's lack of boilerplate is deliberate — it keeps beginners focused on ideas, not scaffolding. Also note Python needs **no semicolons** at the end of lines.

4 Variables and values

4.1 What is a variable?

A **variable** is a *name that refers to a value*. Think of it as a labelled box in which you store something so you can use it again later by its name.

```
age = 20
name = "Ada"
print(name, "is", age, "years old")
```

```
Ada is 20 years old
```

The = sign is **assignment**, not equality. Read `age = 20` as “let *age* *become* 20”, not “age equals 20”. You can change what a variable holds at any time:

```
score = 10
score = score + 5    # take the old value (10), add 5, store it back
print(score)        # => 15
```

That last line shows a very common pattern: using a variable's current value to compute its new value.

4.2 Values have types

Every value has a **type** that determines what you can do with it. The core built-in types you will use constantly:

Type	Python name	Example	What it is
Integer	<code>int</code>	42, -7, 0	whole numbers
Float	<code>float</code>	3.14, -0.5	numbers with a decimal
String	<code>str</code>	"hello", 'A'	text
Boolean	<code>bool</code>	True, False	truth values
None	<code>NoneType</code>	None	"nothing / no value yet"

You can ask Python for a value's type with the `type` function:

```
print(type(42))
print(type(3.14))
print(type("hello"))
print(type(True))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

4.3 Dynamic typing: a key Python idea

In Python you **do not declare** a variable's type. You just assign a value, and the variable takes on that value's type. You can even reassign a different type later (though doing so for no reason is confusing):

```
x = 10          # x is now an int
x = "ten"       # perfectly legal: x is now a str
print(x)
```

```
ten
```

This is called **dynamic typing**. It makes Python quick to write but means some mistakes are only discovered when the program runs.

Python vs JavaScript vs C#: declaring variables

```
# Python – no type, no keyword  
age = 20
```

```
// JavaScript – keyword, still no type  
let age = 20;  
const PI = 3.14159; // const = cannot be reassigned
```

```
// C# – the type is part of the declaration (static typing)  
int age = 20;  
string name = "Ada";
```

- Python and JavaScript are **dynamically typed**: types are checked while the program runs.
- C# is **statically typed**: types are checked *before* running, by the compiler. This catches a whole class of bugs early, at the cost of more typing.
- Python *can* add optional “type hints” (e.g. `age: int = 20`) for documentation and tooling, but they do not change how the program runs.

4.4 Naming variables

Good names make code readable. Rules and conventions in Python:

- Use lowercase words separated by underscores: `student_count`, `total_price`. (This style is called **snake_case**.)
- Names may contain letters, digits, and `_`, but must not *start* with a digit.
- Names are **case-sensitive**: `Age` and `age` are different **variables**.
- Avoid Python’s reserved words (`if`, `for`, `class`, `print` is best avoided too, etc.).

```
# Good  
days_until_exam = 14  
# Legal but unclear – avoid  
d = 14
```

Tip

JavaScript and C# conventionally use `camelCase` (`studentCount`) instead of `snake_case`. The language does not care, but every community has a house style — matching it makes your code easier for others to read.

5 Numbers and arithmetic

Python is a capable calculator out of the box.

```
print(7 + 3)    # addition      => 10
print(7 - 3)    # subtraction  => 4
print(7 * 3)    # multiplication=> 21
print(7 / 3)    # division      => 2.333... (always a float)
print(7 // 3)   # floor division=> 2 (drops the remainder)
print(7 % 3)    # modulo        => 1 (the remainder)
print(7 ** 3)   # exponent      => 343 (7 to the power 3)
```

```
10
4
21
2.3333333333333335
2
1
343
```

Two of these surprise beginners:

- **/** **always** gives a float, even `6 / 2` is `3.0`.
- **//** is **floor division** — it divides and throws away the fractional part.
- **%** (**modulo**) gives the remainder. It is enormously useful: `n % 2 == 0` tests whether `n` is even.

Operator precedence follows normal maths: `**` first, then `*` `/` `//` `%`, then `+` `-`. Use parentheses when in doubt:

```
print(2 + 3 * 4)    # => 14, not 20
print((2 + 3) * 4)   # => 20
```

```
14
20
```

Shorthand for updating a variable, used everywhere:

```
total = 100
total += 10    # same as: total = total + 10
total -= 5     # total = total - 5
total *= 2     # total = total * 2
print(total)   # => 210
```

6 Strings: working with text

A **string** is text. You can write it with single or double quotes (pick one and be consistent); triple quotes span multiple lines.

```
single = 'hello'
double = "hello"
multi = """This string
spans two lines."""
print(multi)
```

```
This string
spans two lines.
```

6.1 Combining and repeating

```
first = "Ada"
last = "Lovelace"
full = first + " " + last      # + joins strings ("concatenation")
print(full)                   # => Ada Lovelace
print("ha" * 3)                # * repeats a string => hahaha
```

```
Ada Lovelace
hahaha
```

6.2 f-strings: the modern way to build text

Putting an **f** before the quotes lets you drop **variables** straight into a string inside {curly braces}. This is the preferred style in modern Python.

```
name = "Ada"
age = 20
print(f"{name} is {age} years old.")
print(f"Next year she will be {age + 1}.")  # you can compute inside braces
```

```
Ada is 20 years old.
Next year she will be 21.
```


6.3 Useful string operations

```
s = "  Data Science  "
print(len(s))          # length, including spaces => 15
print(s.strip())       # remove surrounding whitespace
print(s.upper())       # "  DATA SCIENCE  "
print(s.lower())
print(s.strip().replace("Data", "Computer"))
print("a,b,c".split(",")) # split into a list => ['a', 'b', 'c']
```

```
16
Data Science
  DATA SCIENCE
  data science
Computer Science
['a', 'b', 'c']
```

Strings are **indexed** starting at **0**. You can grab one character or a **slice** (a range):

```
word = "Python"
print(word[0])      # first character => P
print(word[-1])     # last character  => n
print(word[0:3])    # characters 0,1,2 => Pyt
print(word[3:])     # from index 3 to end => hon
```

```
P
n
Pyt
hon
```

⚠ Counting from zero

Almost every programming language counts positions starting at **0**, not 1. So in "Python", **P** is at position 0 and **n** is at position 5. This trips up every beginner at first — it will become second nature.

i Python vs JavaScript vs C#: string formatting

```
print(f"{name} is {age}")          # Python f-string
```

```
console.log(`${name} is ${age}`);  // JS template literal (backticks)
```

```
Console.WriteLine($"{name} is {age}"); // C# interpolated string
```

All three landed on nearly the same idea — embed expressions directly in text — just with different punctuation.

7 Getting input and showing output

`print` sends text out. `input` reads a line the user types in. **input always returns a string**, so convert it if you need a number:

```
name = input("What is your name? ")
age_text = input("How old are you? ")    # e.g. the user types 20
age = int(age_text)                      # convert "20" -> 20
print(f"Hello {name}, next year you'll be {age + 1}.")
```

Forgetting the conversion is a classic bug: `age_text + 1` would error because you cannot add a number to text.

Type conversion functions you will use often:

```
print(int("42"))      # text -> integer  => 42
print(float("3.14"))  # text -> float    => 3.14
print(str(42))         # number -> text   => "42"
print(int(3.9))        # float -> int (truncates) => 3
```

```
42
3.14
42
3
```

8 Booleans, comparisons, and logic

A **Boolean** is either `True` or `False`. They are the answer to yes/no questions and the foundation of all decision-making in code.

Comparison operators produce Booleans:

```
print(5 > 3)      # greater than      => True
print(5 < 3)      # less than         => False
print(5 >= 5)     # greater or equal  => True
print(5 == 5)     # EQUAL? (two equals) => True
print(5 != 3)     # not equal         => True
```

```
True
False
True
True
True
```

 **=** versus **==**

- **= assigns:** `x = 5` puts 5 into x.
- **== compares:** `x == 5` asks “is x equal to 5?” and yields `True/False`.

Mixing these up is the single most common beginner error.

Logical operators combine Booleans. Python spells them as words:

```
sunny = True
warm = False
print(sunny and warm)  # True only if BOTH are true => False
print(sunny or warm)   # True if EITHER is true      => True
print(not sunny)        # flips the value            => False
```

```
False
True
False
```

i Python vs JavaScript vs C#: logical operators

Meaning	Python	JavaScript / C#
and	and	&&
or	or	
not	not	!

Python's use of plain English words is one more reason it reads so smoothly.

9 Making decisions: conditionals

An **if statement** runs a block of code only when a condition is `True`.

```
temperature = 30

if temperature > 25:
    print("It's hot.")
    print("Drink water.")
```

```
It's hot.
Drink water.
```

Two things to notice, both essential to Python:

1. The condition line ends with a **colon** `:`.
2. The code controlled by the **if** is **indented** (four spaces). Indentation is not just style in Python – it is how the language knows which lines belong to the **if**. This is Python's most distinctive feature.

9.1 else and elif

else handles the “otherwise” case. **elif** (“else if”) chains more tests. Python checks them top to bottom and runs the **first** one that is true.

```
score = 72

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
```

```

    grade = "C"
else:
    grade = "F"

print(f"Your grade is {grade}.")

```

Your grade is C.

i Python vs JavaScript vs C#: blocks and indentation

```

# Python – indentation defines the block
if x > 0:
    print("positive")

```

```

// JavaScript – curly braces define the block; indentation is optional style
if (x > 0) {
    console.log("positive");
}

```

```

// C# – also braces, and the condition must be in parentheses
if (x > 0) {
    Console.WriteLine("positive");
}

```

In JS and C# the { } braces mark the block and indentation is merely encouraged. In **Python the indentation IS the block** — get it wrong and the program means something different (or refuses to run). Be consistent: pick four spaces and never mix tabs and spaces.

10 Collections of data

Real programs juggle *many* values at once — a class of students, a column of measurements. Python gives you four core containers. We will meet all four, then focus on the two you will use most: **lists** and **dictionaries**.

Type	Looks like	Ordered?	Changeable?	Typical use
list	[1, 2, 3]	yes	yes	a sequence of items
tuple	(1, 2, 3)	yes	no	a fixed group, e.g. a point

Type	Looks like	Ordered?	Changeable?	Typical use
dict	<code>{"a": 1, "b": 2}</code>	yes*	yes	labelled lookup (key→value)
set	<code>{1, 2, 3}</code>	no	yes	unique items, membership

***Dictionaries** remember insertion order in modern Python.

10.1 Lists

A **list** is an ordered, changeable sequence. Square brackets, items separated by commas.

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])      # first item    => apple
print(fruits[-1])     # last item     => cherry
print(len(fruits))    # how many      => 3
```

```
apple
cherry
3
```

Lists are **mutable** — you can change them after creation:

```
fruits.append("date")    # add to the end
fruits[1] = "blueberry"  # replace item at index 1
fruits.remove("apple")   # remove by value
print(fruits)            # => ['blueberry', 'cherry', 'date']
```

```
['blueberry', 'cherry', 'date']
```

Slicing works just like with strings:

```
numbers = [10, 20, 30, 40, 50]
print(numbers[1:4])    # items 1,2,3  => [20, 30, 40]
print(numbers[:2])     # first two    => [10, 20]
print(numbers[::2])    # every 2nd    => [10, 30, 50]
```

```
[20, 30, 40]
[10, 20]
[10, 30, 50]
```

A few **list** workhorses:

```

nums = [3, 1, 4, 1, 5, 9, 2]
print(sum(nums))      # => 25
print(max(nums))      # => 9
print(min(nums))      # => 1
print(sorted(nums))   # a new sorted list => [1, 1, 2, 3, 4, 5, 9]
print(len(nums))      # => 7

```

```

25
9
1
[1, 1, 2, 3, 4, 5, 9]
7

```

10.2 Tuples

A **tuple** is like a **list** but **immutable** — once created it cannot change. Use it for a fixed group of values that belong together, such as coordinates.

```

point = (3, 4)
x, y = point          # "unpacking": x becomes 3, y becomes 4
print(x, y)
# point[0] = 99        # this would raise an error – tuples cannot be changed

```

```

3 4

```

10.3 Dictionaries

A **dictionary** stores **key** → **value** pairs. Instead of looking things up by numeric position, you look them up by a meaningful key. This is one of the most useful structures in all of programming.

```

student = {
    "name": "Ada",
    "age": 20,
    "major": "Computer Science",
}
print(student["name"])    # look up by key => Ada
student["age"] = 21       # update a value
student["year"] = 2       # add a new pair
print(student)

```

```

Ada
{'name': 'Ada', 'age': 21, 'major': 'Computer Science', 'year': 2}

```

Looping over a dictionary gives you its keys and values:

```
prices = {"apple": 0.5, "banana": 0.3, "cherry": 2.0}
for fruit, price in prices.items():
    print(f"{fruit} costs ${price:.2f}")
```

```
apple costs $0.50
banana costs $0.30
cherry costs $2.00
```

10.4 Sets

A **set** holds only **unique** items and is great for membership tests and removing duplicates.

```
visitors = ["Ana", "Bo", "Ana", "Cy", "Bo"]
unique = set(visitors)
print(unique)                # duplicates gone
print("Ana" in unique)       # fast membership test => True
print(len(unique))           # => 3
```

```
{'Bo', 'Ana', 'Cy'}
True
3
```

i Python vs JavaScript vs C#: the everyday containers

Python	JavaScript	C#
list	Array	List<T> / array
dict	Object / Map	Dictionary<K,V>
set	Set	HashSet<T>
tuple	(array, by convention)	Tuple / ValueTuple

The big difference: a C# `List<int>` may hold **only integers**, decided up front. A Python `list` can hold anything — even a mix — because Python is dynamically typed. Convenient, but it puts the responsibility on *you* to keep your data sensible.

11 Repeating work: loops

Computers shine at doing the same thing many times. A **loop** repeats a block of code. Python has two kinds.

11.1 The **for** loop — “do this for each item”

A **for** loop walks through a collection, one item at a time.

```
for fruit in ["apple", "banana", "cherry"]:
    print(f"I like {fruit}")
```

```
I like apple
I like banana
I like cherry
```

To repeat something a fixed number of times, loop over **range**:

```
for i in range(5):          # i takes values 0, 1, 2, 3, 4
    print(i)
```

```
0
1
2
3
4
```

range is flexible:

```
print(list(range(5)))        # [0, 1, 2, 3, 4]    stop before 5
print(list(range(2, 8)))     # [2, 3, 4, 5, 6, 7] start..stop
print(list(range(0, 10, 2))) # [0, 2, 4, 6, 8]    with a step
```

```
[0, 1, 2, 3, 4]
[2, 3, 4, 5, 6, 7]
[0, 2, 4, 6, 8]
```

A common pattern — accumulate a result while looping:

```
numbers = [4, 8, 15, 16, 23, 42]
total = 0
for n in numbers:
    total += n          # add each number to the running total
print(f"Sum is {total}") # => 108
```

```
Sum is 108
```

11.2 The **while** loop — “keep going as long as...”

A **while** loop repeats *as long as a condition stays true*. Use it when you do not know in advance how many repetitions you need.

```
countdown = 3
while countdown > 0:
    print(countdown)
    countdown -= 1      # MUST move toward stopping
print("Lift off!")
```

```
3
2
1
Lift off!
```

⚠ Infinite loops

If the condition in a **while** loop never becomes false, the loop runs forever and your program hangs. Always make sure something inside the loop changes the condition — here, `countdown -= 1`. If you ever get stuck in one, press **Ctrl + C** to interrupt.

11.3 **break** and **continue**

- **break** exits the loop immediately.
- **continue** skips the rest of the current pass and moves to the next item.

```
for n in range(1, 11):
    if n == 7:
        break          # stop entirely when we hit 7
    if n % 2 == 0:
        continue       # skip even numbers
    print(n)           # prints 1, 3, 5
```

```
1
3
5
```

11.4 When to use which loop

- Use **for** when you have a collection or a known number of repetitions (the common case — reach for this first).
- Use **while** when you must repeat until some condition changes and you cannot count the iterations ahead of time.

i Python vs JavaScript vs C#: loops

```
# Python – iterate directly over items
for fruit in fruits:
    print(fruit)

for i in range(5):
    print(i)
```

```
// JavaScript
for (const fruit of fruits) { console.log(fruit); }
for (let i = 0; i < 5; i++) { console.log(i); }
```

```
// C#
foreach (var fruit in fruits) { Console.WriteLine(fruit); }
for (int i = 0; i < 5; i++) { Console.WriteLine(i); }
```

The classic `for (i = 0; i < n; i++)` form from C#/JS exists in Python too via `range`, but Python prefers iterating over the items themselves — it is cleaner and avoids off-by-one mistakes.

11.5 List comprehensions: a Python superpower

A **list comprehension** builds a new **list** from an old one in a single readable line. It is so idiomatic that you will see it everywhere.

```
numbers = [1, 2, 3, 4, 5]

squares = [n * n for n in numbers]          # transform every item
print(squares)                             # => [1, 4, 9, 16, 25]

evens = [n for n in numbers if n % 2 == 0]  # keep only some items
print(evens)                               # => [2, 4]
```

```
[1, 4, 9, 16, 25]
[2, 4]
```

Read the first one as: “*n * n, for each n in numbers.*” It does the same job as a `for` loop with `.append()`, but shorter and clearer.

12 Functions

A **function** is a named, reusable block of code. You **define** it once, then **call** it as many times as you like. Functions are how we avoid repeating ourselves and how we break a big problem into small, nameable pieces.

12.1 Defining and calling

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Ada")          # calling the function  
greet("Grace")
```

```
Hello, Ada!  
Hello, Grace!
```

- `def` starts the definition.
- `name` is a **parameter** — a placeholder for whatever value you pass in.
- The body is indented, just like an `if` or a loop.

12.2 Returning a value

Most useful functions **return** a result with the `return` keyword, so the caller can use it.

```
def add(a, b):  
    return a + b  
  
result = add(3, 4)      # result becomes 7  
print(result)  
print(add(10, 20) * 2)  # use the returned value directly
```

```
7  
60
```

💡 print versus return

- `print` shows something on screen — then it is gone.
- `return` hands a value *back to your program* so you can store or reuse it.

A function that only prints cannot be used in further calculations. Beginners often confuse the two: prefer `return` for anything you will use again.

12.3 Default values and named arguments

Parameters can have **defaults**, making them optional. You can also pass arguments **by name** for clarity.

```
def power(base, exponent=2):    # exponent defaults to 2
    return base ** exponent

print(power(5))                # uses default => 25
print(power(5, 3))            # => 125
print(power(base=2, exponent=10)) # named for readability => 1024
```

```
25
125
1024
```

12.4 Scope: where variables live

A variable created **inside** a function exists only inside that function. This is called **local scope**, and it is a feature: it keeps functions from accidentally clobbering each other's **variables**.

```
def compute():
    secret = 42    # local: only visible inside compute()
    return secret

print(compute())    # => 42
# print(secret)     # would error: 'secret' is not defined out here
```

```
42
```

12.5 A worked example

Bundling a small task into a well-named function makes the rest of your code read like plain English:

```
def is_prime(n):
    """Return True if n is a prime number."""
    if n < 2:
        return False
    for divisor in range(2, int(n ** 0.5) + 1):
        if n % divisor == 0:
            return False    # found a factor → not prime
    return True
```

```
primes = [n for n in range(2, 30) if is_prime(n)]
print(primes)
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

That text in triple quotes just under `def` is a **docstring** — a short description of what the function does. It is good manners and shows up in help.

i Python vs JavaScript vs C#: functions

```
# Python
def add(a, b):
    return a + b
```

```
// JavaScript
function add(a, b) { return a + b; }
const add = (a, b) => a + b;           // "arrow function" shorthand
```

```
// C# – return type and parameter types are declared
int Add(int a, int b) { return a + b; }
```

Again: Python and JS do not state the types; C# requires `int` both for the parameters and the return value. C#'s compiler will refuse `Add("x", 3)` before the program ever runs; in Python that mistake only surfaces at run time.

13 When things go wrong: errors and exceptions

Errors are normal — every programmer meets them constantly. Learning to **read** them is a core skill. Python **errors** come in two flavours.

Syntax errors mean your code is not valid Python (a missing colon, an unmatched bracket). Python refuses to run it at all:

```
if x > 5          # SyntaxError: missing colon
    print("big")
```

Exceptions happen while the program is running, when something legal-looking turns out to be impossible:

```
print(10 / 0)      # ZeroDivisionError
```

```

ZeroDivisionError: division by zero
[31m-----[39m
[31mZeroDivisionError[39m                                Traceback (most recent
call last)
[36mCell[39m[36m [39m[32mIn[39][39m[32m, line 1[39m
[32m---> [39m[32m1[39m      print([32m10[39m      /      [32m0[39m)
[38;5;66;03m# ZeroDivisionError[39;00m

[31mZeroDivisionError[39m: division by zero

```

Read the last line of an error message first — it names the problem (`ZeroDivisionError: division by zero`). The lines above show *where* it happened (the “traceback”).

13.1 Handling exceptions with `try / except`

Sometimes you expect something might fail and want to recover gracefully instead of crashing.

```

def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print("Cannot divide by zero – returning None instead.")
        return None

print(safe_divide(10, 2))    # => 5.0
print(safe_divide(10, 0))    # prints the warning, returns None

```

```

5.0
Cannot divide by zero – returning None instead.
None

```

This pattern is essential for real programs that deal with messy input: read a file that might not exist, parse a number the user might mistype, call a network service that might be down.

Python vs JavaScript vs C#: error handling

The idea is shared across languages, only the keywords differ:

Python	JavaScript / C#
<code>try: / except:</code>	<code>try { } catch { }</code>
<code>raise</code>	<code>throw</code>
<code>finally:</code>	<code>finally { }</code>

14 Reading and writing files

Programs become useful when they work with data on disk. The modern, safe way to open a file is the `with` block, which closes the file for you automatically.

```
# Writing text to a file
with open("notes.txt", "w") as f:      # "w" = write (overwrites!)
    f.write("First line\n")
    f.write("Second line\n")

# Reading it back
with open("notes.txt", "r") as f:      # "r" = read
    contents = f.read()
print(contents)
```

To process a file line by line — common for data:

```
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())           # strip() removes the trailing newline
```

Tip

For real **datasets** (spreadsheets, CSV files) you will rarely use raw `open`. Instead you will hand the file to the **Pandas** library, which understands tabular data — see Section 18.

15 A glimpse of objects and classes

So far we have used data (lists, dicts) and behaviour (functions) separately. A **class** bundles them together: it is a blueprint that packages related data (**attributes**) and the functions that act on them (**methods**). An individual thing built from the blueprint is an **object** (or *instance*).

You have already been using objects without knowing it: a string is an object, and `"hello".upper()` calls the string's `upper` method.

```
class BankAccount:
    def __init__(self, owner, balance=0):  # the "constructor"
        self.owner = owner                # attribute
        self.balance = balance            # attribute

    def deposit(self, amount):              # a method
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds.")
        else:
            self.balance -= amount

# Build two independent objects from the one blueprint
acc = BankAccount("Ada", 100)
acc.deposit(50)
acc.withdraw(30)
print(f"{acc.owner} has ${acc.balance}")    # => Ada has $120
```

```
Ada has $120
```

A few terms:

- `__init__` is the **constructor** — it runs automatically when you create an object, setting up its initial data.
- `self` refers to *the particular object* a method is working on. Every method's first parameter is `self`.

Object-oriented programming is a large topic. For a first course it is enough to **recognise** classes and feel comfortable *using* objects and their methods, since the data libraries below are built from them.

i Python vs JavaScript vs C#: classes

All three are object-oriented and share the vocabulary of classes, objects, attributes, and methods. C# leans heavily on classes for almost everything; Python lets you write plenty of useful programs with just functions and reach for classes only when they genuinely help. Start simple.

16 Modules, packages, and the wider ecosystem

You do not have to write everything yourself. A **module** is a file of reusable code; a **package** is a collection of modules. Python ships with a large “standard library”, and a vast world of third-party packages is one install away.

16.1 Importing

```
import math
print(math.sqrt(16))      # => 4.0
print(math.pi)            # => 3.14159...

from random import randint
print(randint(1, 6))      # a dice roll – import just one name
```

```
4.0
3.141592653589793
1
```

16.2 Installing third-party packages with **pip**

pip is Python’s package installer. You run it in a terminal, not inside a Python program:

```
pip install numpy pandas matplotlib
```

These three packages turn Python into a data-analysis powerhouse, and they are the subject of the rest of this reader.

i Python vs JavaScript vs C#: getting libraries

Python	JavaScript	C#
pip	npm	NuGet

Same idea everywhere: a command that downloads community-written packages so you can stand on the shoulders of others.

17 Numerical computing with NumPy

numpy (Numerical Python) introduces the **array**: a grid of numbers that you can compute on all at once, far faster and more conveniently than a plain **list**. It is the foundation under almost every scientific and **AI** library in Python.

```
import numpy as np          # "np" is the universal nickname

a = np.array([1, 2, 3, 4, 5])
print(a)
print(a + 10)              # adds 10 to EVERY element at once
print(a * 2)               # multiplies every element
print(a.mean())            # average => 3.0
print(a.sum())             # => 15
```

```
[1 2 3 4 5]
[11 12 13 14 15]
[ 2  4  6  8 10]
3.0
15
```

The key idea is **vectorisation**: an operation written once applies to the whole array, with no loop. Compare:

```
# The slow, manual way (a loop)
nums = [1, 2, 3, 4, 5]
doubled = [n * 2 for n in nums]

# The NumPy way – same result, but it scales to millions of numbers efficiently
doubled_np = np.array(nums) * 2
print(doubled, doubled_np)
```

```
[2, 4, 6, 8, 10] [ 2  4  6  8 10]
```

NumPy also generates and reshapes data easily:

```
print(np.zeros(3))          # => [0. 0. 0.]
print(np.arange(0, 10, 2))  # like range, but an array => [0 2 4 6 8]
print(np.linspace(0, 1, 5)) # 5 evenly spaced values 0..1
grid = np.arange(6).reshape(2, 3) # a 2x3 grid
print(grid)
```

```
[0. 0. 0.]
[0 2 4 6 8]
[0.  0.25 0.5  0.75 1.  ]
[[0 1 2]
 [3 4 5]]
```

18 Tabular data with Pandas

`pandas` is the workhorse of data analysis. Its central object is the **DataFrame** — a table with named columns, like a spreadsheet you can program.

```
import pandas as pd

# Normally you'd load a file with pd.read_csv("data.csv").
# Here we build a small table by hand so the example is self-contained.
df = pd.DataFrame({
    "name":  ["Ada", "Bo", "Cy", "Dee", "Eli"],
    "age":   [20, 22, 19, 21, 23],
    "score": [88, 75, 95, 60, 82],
    "major": ["CS", "Math", "CS", "Physics", "Math"],
})
df
```

	name	age	score	major
0	Ada	20	88	CS
1	Bo	22	75	Math
2	Cy	19	95	CS
3	Dee	21	60	Physics
4	Eli	23	82	Math

18.1 Looking at your data

The first thing you always do with a new **dataset**:

```
print(df.shape)          # (rows, columns) => (5, 4)
print(df.columns.tolist())
df.head(3)               # the first 3 rows
```

```
(5, 4)
['name', 'age', 'score', 'major']
```

	name	age	score	major
0	Ada	20	88	CS
1	Bo	22	75	Math
2	Cy	19	95	CS

```
df.describe()          # quick statistics for the numeric columns
```

	age	score
count	5.000000	5.000000
mean	21.000000	80.000000
std	1.581139	13.397761
min	19.000000	60.000000
25%	20.000000	75.000000
50%	21.000000	82.000000
75%	22.000000	88.000000
max	23.000000	95.000000

18.2 Selecting columns and filtering rows

```
print(df["score"].mean())          # average score
print(df[["name", "score"]])      # just two columns

# Filtering: keep only rows where a condition is true (a "boolean mask")
top = df[df["score"] >= 80]
print(top[["name", "score"]])
```

```
80.0
  name  score
0  Ada     88
1   Bo     75
2   Cy     95
3  Dee     60
4  Eli     82
  name  score
0  Ada     88
2   Cy     95
4  Eli     82
```

That `df["score"] >= 80` produces a column of `True/False`, and indexing the DataFrame with it keeps only the `True` rows — the same Boolean idea from Section 8, applied to a whole table at once.

18.3 Creating columns and grouping

```
# Add a computed column
df["passed"] = df["score"] >= 70
print(df[["name", "score", "passed"]])
```

	name	score	passed
0	Ada	88	True
1	Bo	75	True
2	Cy	95	True
3	Dee	60	False
4	Eli	82	True

```
# "Group by" a category and summarise each group – the heart of data analysis
by_major = df.groupby("major")["score"].mean()
print(by_major)
```

```
major
CS      91.5
Math    78.5
Physics 60.0
Name: score, dtype: float64
```

`groupby` answers questions like “*what is the average score within each major?*” It splits the table into groups, applies a calculation to each, and combines the answers — a pattern you will use constantly.

18.4 Sorting

```
df.sort_values("score", ascending=False)
```

	name	age	score	major	passed
2	Cy	19	95	CS	True
0	Ada	20	88	CS	True
4	Eli	23	82	Math	True
1	Bo	22	75	Math	True
3	Dee	21	60	Physics	False

💡 The typical data-analysis workflow

1. **Load** the data (`pd.read_csv()`).
2. **Inspect** it (`.head()`, `.describe()`, `.shape`).
3. **Clean** it (handle missing values, fix types).
4. **Transform** it (filter, add columns, `groupby`).
5. **Visualise** it (next section).
6. **Conclude** — state what you found.

You will repeat this loop on every **dataset** you ever touch.

19 Visualising data with Matplotlib

A picture makes patterns jump out that a table hides. `matplotlib` is Python's foundational plotting library. **Pandas** can even call it for you.

19.1 A line plot

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)    # 100 points from 0 to 10
y = x ** 2

plt.figure()
plt.plot(x, y)
plt.title("y = x squared")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)
plt.show()
```

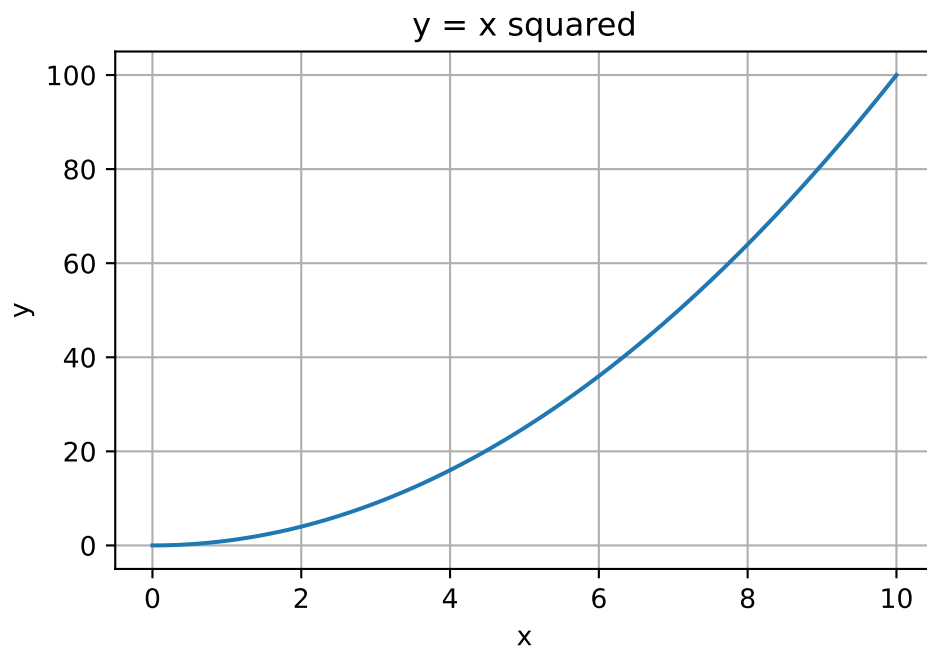


Figure 1: A simple line plot of $y = x$ squared.

Every good chart shares the same anatomy: a **title**, **axis labels**, and the data itself. Label your axes — an unlabelled **plot** is a puzzle, not a result.

19.2 A bar chart from our DataFrame

```
plt.figure()
plt.bar(df["name"], df["score"], color="steelblue")
plt.title("Scores by student")
plt.xlabel("Student")
plt.ylabel("Score")
plt.ylim(0, 100)
plt.show()
```

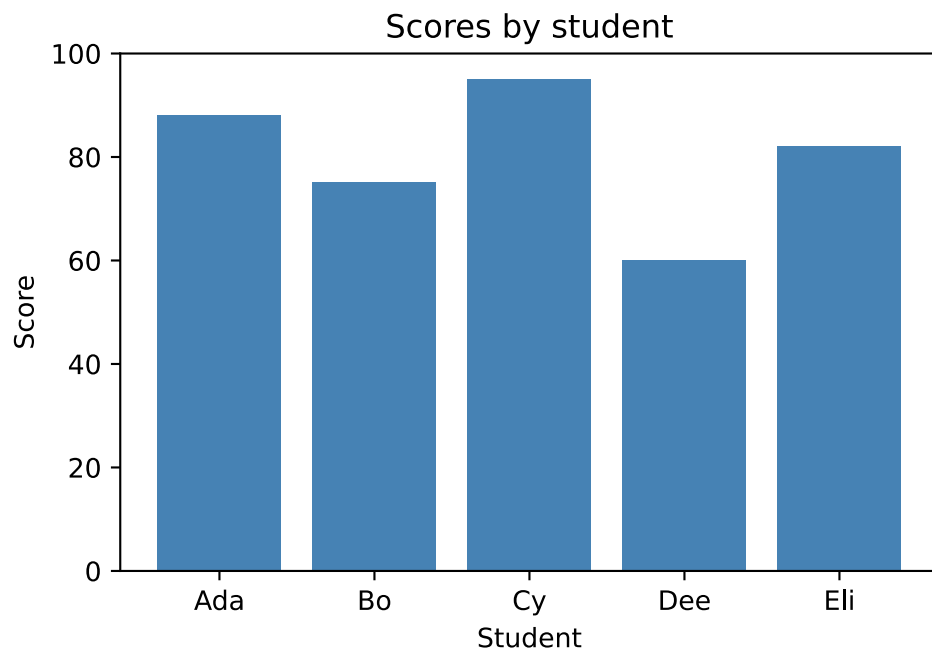



Figure 2: Each student's score.

19.3 A scatter plot — looking for relationships

```
plt.figure()
plt.scatter(df["age"], df["score"], s=80, color="darkorange")
plt.title("Age vs. Score")
plt.xlabel("Age")
plt.ylabel("Score")
plt.show()
```

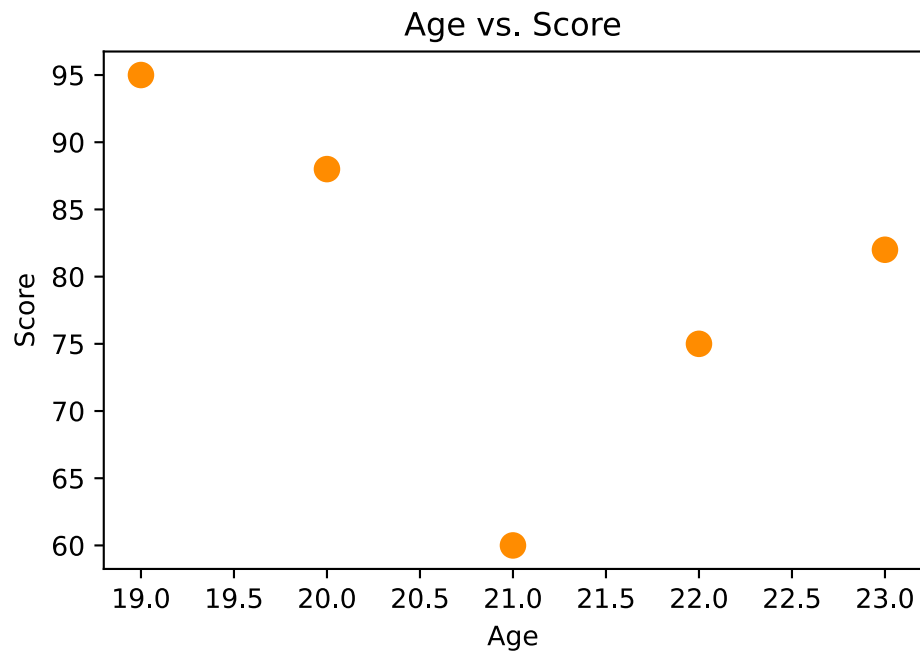


Figure 3: Is there a relationship between age and score?

19.4 A histogram — the shape of a distribution

```
rng = np.random.default_rng(42)          # seeded, so results are reproducible
exam_scores = rng.normal(loc=70, scale=10, size=1000)

plt.figure()
plt.hist(exam_scores, bins=30, color="seagreen", edgecolor="white")
plt.title("Exam score distribution")
plt.xlabel("Score")
plt.ylabel("How many students")
plt.show()
```

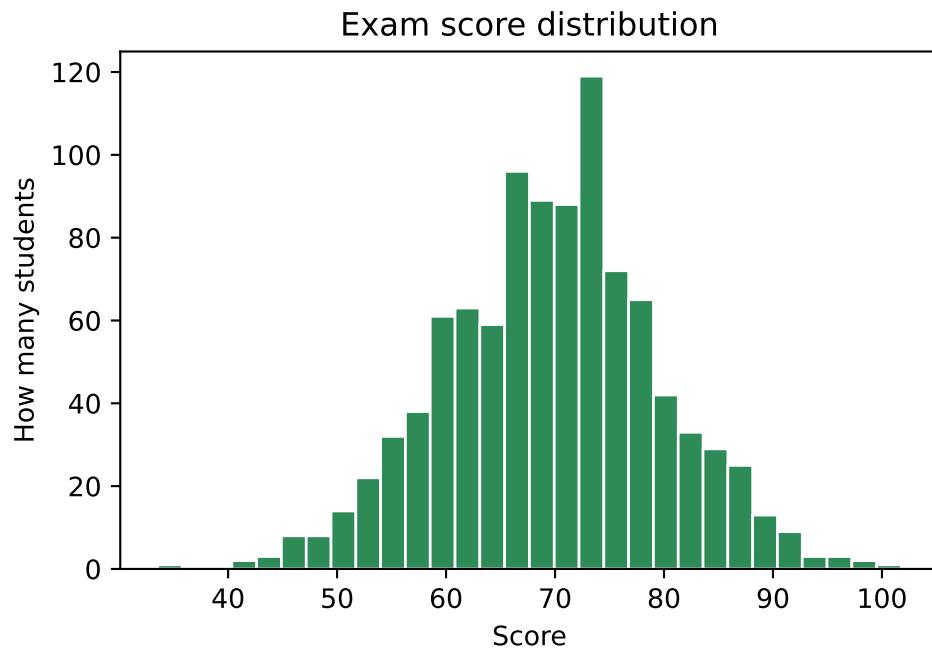


Figure 4: Distribution of 1000 random exam scores.

A histogram buckets values into ranges and shows how many fall in each — the go-to chart for “what does my data look like?”

20 A complete mini-analysis

Let us put the whole toolkit together on one small, made-up **dataset**, following the workflow from Section 18. The goal: **does study time relate to exam score?**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 1. Create reproducible synthetic data -----
rng = np.random.default_rng(0)
hours = rng.uniform(0, 10, size=60)           # hours studied
noise = rng.normal(0, 8, size=60)             # random variation
scores = 50 + 4.5 * hours + noise              # a real trend + noise
scores = np.clip(scores, 0, 100)              # keep within 0..100

data = pd.DataFrame({"hours": hours, "score": scores})

# 2. Inspect -----
print(data.describe().round(2))
```

```

# 3. Quantify the relationship -----
correlation = data["hours"].corr(data["score"])
print(f"\nCorrelation between hours and score: {correlation:.2f}")

# Fit a simple straight line: score ≈ slope * hours + intercept
slope, intercept = np.polyfit(data["hours"], data["score"], 1)
print(f"Best-fit line: score = {slope:.1f} * hours + {intercept:.1f}")

# 4. Visualise -----
plt.figure()
plt.scatter(data["hours"], data["score"], alpha=0.7, label="students")
line_x = np.array([0, 10])
plt.plot(line_x, slope * line_x + intercept, color="red", label="trend")
plt.title("More study time tends to mean higher scores")
plt.xlabel("Hours studied")
plt.ylabel("Exam score")
plt.legend()
plt.show()

```

	hours	score
count	60.00	60.00
mean	5.05	73.35
std	2.99	15.46
min	0.03	35.78
25%	2.62	61.66
50%	5.43	75.30
75%	7.44	83.92
max	9.97	100.00

Correlation between hours and score: 0.85
 Best-fit line: score = 4.4 * hours + 51.1

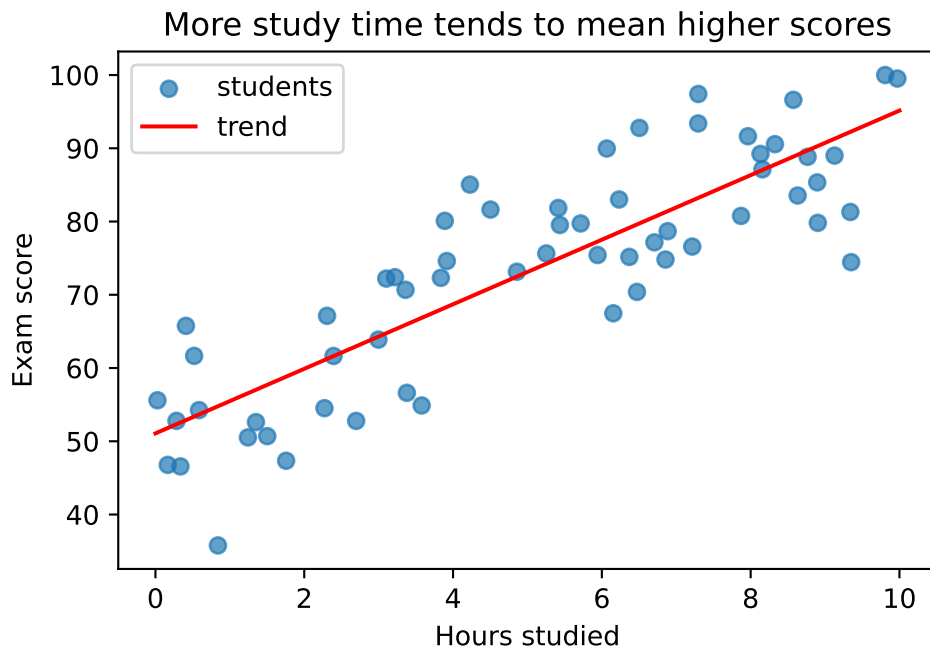


Figure 5: Study hours vs. exam score, with a fitted trend line.

Reading the result. A correlation near $+0.8$ and an upward red line both say the same thing: students who studied more *tended* to score higher, by roughly 4–5 points per extra hour. Note the careful wording — *tended to*. Correlation is not proof of cause, and this is invented data. Stating conclusions honestly and with appropriate caution is itself a skill this course aims to build.

21 Programming in the age of AI

You are learning to code at a moment when **AI** assistants (like Claude, GitHub Copilot, ChatGPT) can write code from a plain-English description. This changes *how* you work, but it makes the fundamentals in this reader **more** important, not less. Here is how to use these tools well.

21.1 What AI assistants are good at

- **Boilerplate and syntax recall:** “How do I read a CSV in **pandas**?”
- **Explaining code or an error message** you do not understand.
- **First drafts** of a function you then check and refine.
- **Translating** an idea between languages (“do this Python in C#”).

21.2 Why the fundamentals still matter

An **AI** assistant is a confident collaborator that is *sometimes wrong*. It can produce code that looks plausible but is subtly incorrect, inefficient, or solves the wrong problem. To use it safely you must be able to:

1. **Read** the code it gives you (every concept in this reader).
2. **Test** it — run it, try edge cases, check the output is actually right.

3. **Judge** whether the approach makes sense for your problem.

If you cannot read the code, you cannot tell good output from confident nonsense. The skills in this document are exactly the skills needed to *supervise* an **AI**, not compete with it.

21.3 How to ask well

Vague prompts get vague code. Good prompts are specific:

✗ “Write code to analyse my data.”

✓ “I have a **pandas** DataFrame **df** with columns **date** (string) and **sales** (float). Write a function that returns total sales per month, and explain each step.”

Give the assistant the **context** (your data shape), the **goal**, and ask it to **explain**. Then read the explanation and verify the result yourself.

21.4 A healthy workflow

1. **Try first**. Attempt the problem yourself; you learn by struggling a little.
2. **Ask for help** when stuck — for an explanation, not just an answer.
3. **Understand** every line before you keep it. If you cannot explain it, you are not ready to rely on it.
4. **Verify** by running and testing. Never paste code you have not checked into anything that matters.
5. **Cite and be honest** about what was **AI**-assisted, especially in coursework — follow your instructor’s policy.

! The bottom line

AI turns programming from *typing* into *deciding and verifying*. Those higher skills — framing a problem, reading a solution critically, testing it, judging whether it is correct — are built on exactly the fundamentals taught here. Learn them well, and **AI** becomes a force multiplier rather than a crutch.

22 Quick reference

A one-page cheat sheet of the most-used Python.

```
# --- Variables & types -----  
x = 10                # int  
pi = 3.14             # float  
name = "Ada"          # str  
flag = True           # bool  
nothing = None        # absence of a value
```

```

# --- Strings -----
f"{name} is {x}"      # f-string
name.upper()          # method call
len(name)             # length
name[0]               # indexing (from 0)

# --- Collections -----
nums = [1, 2, 3]       # list    (ordered, changeable)
point = (3, 4)         # tuple   (fixed)
person = {"name": "Ada", "age": 20} # dict   (key -> value)
unique = {1, 2, 3}     # set     (unique items)

# --- Conditionals -----
if x > 0:
    ...
elif x == 0:
    ...
else:
    ...

# --- Loops -----
for item in nums:
    ...
for i in range(5):
    ...
while x > 0:
    x -= 1

# --- Comprehension -----
squares = [n * n for n in nums]
evens = [n for n in nums if n % 2 == 0]

# --- Functions -----
def add(a, b=0):
    return a + b

# --- Errors -----
try:
    risky()
except SomeError:
    handle()

```

```
# --- Files -----
with open("file.txt") as f:
    text = f.read()

# --- Data stack -----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("data.csv")
df.describe()
df.groupby("col")["val"].mean()
plt.plot(x, y); plt.show()
```

22.1 Common beginner pitfalls

Pitfall	Fix
Using <code>=</code> where you meant <code>==</code>	<code>=</code> assigns, <code>==</code> compares
Forgetting the <code>:</code> after <code>if/for/def</code>	Every block header ends with a colon
Mixing tabs and spaces for indentation	Pick 4 spaces and stay consistent
Counting from 1 instead of 0	The first item is index 0
<code>input()</code> returns text, not a number	Wrap it: <code>int(input(...))</code>
Expecting <code>print</code> to return a value	Use <code>return</code> to pass values back
Infinite <code>while</code> loop	Ensure the condition eventually goes false

23 Where to go next

- **Practice** on small problems daily. Fluency comes from reps, not reading.
- Official Python tutorial — thorough and free.
- Automate the Boring Stuff with Python — beginner-friendly, free online.
- For data: the Pandas and Matplotlib documentation, each with galleries of examples.
- Keep a personal “snippets” file of solutions you understand, and revisit this reader whenever a concept feels shaky.

💡 One last piece of advice

You learn programming the way you learn a language or an instrument: by *doing*, making mistakes, and fixing them. Read less, type more. Break things on purpose to see what the error says. Every expert was once exactly where you are now.